

HOMEWORK 5

Gaurav Batra
gbatra3@wisc.edu

Instructions: Use this L^AT_EX file as a template to develop your homework. Submit your homework on time as a single PDF file to **Gradescope**. You can choose any programming language (i.e., Python, R, or MATLAB). Please check Piazza for updates about the homework. *Late submissions and hand-written (or non-typed) homework will not be accepted and will receive 0 credit.*

1 Convolutional Neural Networks (27 points, Extra Credits)

Please complete the CNN classifier for the MNIST dataset using the provided code in `mnist_cnn.py`, and complete the following two tasks. This problem requires using Python and the PyTorch package.

Hint: i) You may use internet resources, e.g., a tutorial of PyTorch, to help you solve these questions; ii) You may use Google Colab (<https://colab.research.google.com/>) to debug the code using GPUs.

1.1 Please paste your completion to this question below. You can insert your completions in the template provided. (15 points, Extra Credits)

```
1  import argparse
2  import torch
3  import torch.nn as nn
4  import torch.nn.functional as F
5  import torch.optim as optim
6  from torchvision import datasets, transforms
7  from torch.optim.lr_scheduler import StepLR
8
9
10 class Net(nn.Module):
11     def __init__(self):
12         super(Net, self).__init__()
13         #####
14         # Define the neural network architecture with these specifications:
15         # 1. First convolutional layer (conv1):
16         #   - input channels: 1 (grayscale image)
17         #   - output channels: 32
18         #   - kernel size: 3
19         #   - stride: 1
20         self.conv1 = nn.Conv2d(1, 32, 3, 1)
21
22         # 2. Second convolutional layer (conv2):
23         #   - input channels: 32 (from conv1)
24         #   - output channels: 64
25         #   - kernel size: 3
26         #   - stride: 1
27         self.conv2 = nn.Conv2d(32, 64, 3, 1)
```

```

29     # 3. First dropout layer (dropout1):
30     #     - dropout rate: 0.25
31     self.dropout1 = nn.Dropout2d(0.25)
32
33     # 4. Second dropout layer (dropout2):
34     #     - dropout rate: 0.5
35     self.dropout2 = nn.Dropout2d(0.5)
36
37     # 5. First fully connected layer (fc1):
38     #     - input features: 9216 (64 * 12 * 12)
39     #     - output features: 128
40     self.fc1 = nn.Linear(64 * 12 * 12, 128)
41
42     # 6. Second fully connected layer (fc2):
43     #     - input features: 128
44     #     - output features: 10 (number of classes)
45     self.fc2 = nn.Linear(128, 10)
46     #####
47
48     def forward(self, x):
49         #####
50         # Implement the forward pass with these exact steps:
51         # 1. Apply conv1 and ReLU activation
52         x = F.relu(self.conv1(x))
53         # 2. Apply conv2 and ReLU activation
54         x = F.relu(self.conv2(x))
55         # 3. Apply max pooling with kernel size 2
56         x = F.max_pool2d(x, 2)
57         # 4. Apply dropout1
58         x = self.dropout1(x)
59         # 5. Flatten the tensor (hint: use torch.flatten(x, 1))
60         x = torch.flatten(x, 1)
61         # 6. Apply fc1 and ReLU activation
62         x = F.relu(self.fc1(x))
63         # 7. Apply dropout2
64         x = self.dropout2(x)
65         # 8. Apply fc2
66         x = self.fc2(x)
67         # 9. Apply log_softmax with dim=1
68         return F.log_softmax(x, dim=1)
69         #####
70
71
72     def train(args, model, device, train_loader, optimizer, epoch):
73         model.train()
74         for batch_idx, (data, target) in enumerate(train_loader):
75             data, target = data.to(device), target.to(device)
76             #####
77             # Implement the training step:
78             # 1. Zero the gradients with optimizer.zero_grad()
79             optimizer.zero_grad()
80             # 2. Perform forward pass through the model
81             output = model(data)
82             # 3. Calculate the loss using F.nll_loss(output, target)
83             loss = F.nll_loss(output, target)
84             # 4. Perform backward pass with loss.backward()
85             loss.backward()
86             # 5. Update weights with optimizer.step()

```

```

87     optimizer.step()
88     #####
89
90     if batch_idx % args.log_interval == 0:
91         print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
92             epoch, batch_idx * len(data), len(train_loader.dataset),
93             100. * batch_idx / len(train_loader), loss.item()))
94         if args.dry_run:
95             break
96
97
98 def test(model, device, test_loader):
99     model.eval()
100     test_loss = 0
101     correct = 0
102     with torch.no_grad():
103         for data, target in test_loader:
104             data, target = data.to(device), target.to(device)
105             #####
106             # Implement the testing step:
107             # 1. Perform forward pass through the model
108             output = model(data)
109             # 2. Calculate and accumulate loss using F.nll_loss(output, target,
110             ↪ reduction='sum').item()
111             test_loss += F.nll_loss(output, target, reduction='sum').item()
112             # 3. Get predictions using output.argmax(dim=1, keepdim=True)
113             pred = output.argmax(dim=1, keepdim=True)
114             # 4. Calculate number of correct predictions using
115             ↪ pred.eq(target.view_as(pred)).sum().item()
116             correct += pred.eq(target.view_as(pred)).sum().item()
117             #####
118
119     test_loss /= len(test_loader.dataset)
120
121     print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%)\n'.format(
122         test_loss, correct, len(test_loader.dataset),
123         100. * correct / len(test_loader.dataset)))
124
125
126 def main():
127     # Training settings
128     parser = argparse.ArgumentParser(description='PyTorch MNIST Example')
129     parser.add_argument('--batch-size', type=int, default=64, metavar='N',
130                         help='input batch size for training (default: 64)')
131     parser.add_argument('--test-batch-size', type=int, default=1000, metavar='N',
132                         help='input batch size for testing (default: 1000)')
133     parser.add_argument('--epochs', type=int, default=14, metavar='N',
134                         help='number of epochs to train (default: 14)')
135     parser.add_argument('--lr', type=float, default=1.0, metavar='LR',
136                         help='learning rate (default: 1.0)')
137     parser.add_argument('--gamma', type=float, default=0.7, metavar='M',
138                         help='Learning rate step gamma (default: 0.7)')
139     parser.add_argument('--no-cuda', action='store_true', default=False,
140                         help='disables CUDA training')
141     parser.add_argument('--no-mps', action='store_true', default=False,
142                         help='disables macOS GPU training')
143     parser.add_argument('--dry-run', action='store_true', default=False,
144                         help='quickly check a single pass')

```

```

143 parser.add_argument('--seed', type=int, default=1, metavar='S',
144                     help='random seed (default: 1)')
145 parser.add_argument('--log-interval', type=int, default=10, metavar='N',
146                     help='how many batches to wait before logging training
147                          ↪ status')
147 parser.add_argument('--save-model', action='store_true', default=False,
148                     help='For Saving the current Model')
149 args, _ = parser.parse_known_args()
150 use_cuda = not args.no_cuda and torch.cuda.is_available()
151 use_mps = not args.no_mps and torch.backends.mps.is_available()
152
153 torch.manual_seed(args.seed)
154
155 if use_cuda:
156     device = torch.device("cuda")
157 elif use_mps:
158     device = torch.device("mps")
159 else:
160     device = torch.device("cpu")
161
162 train_kwargs = {'batch_size': args.batch_size}
163 test_kwargs = {'batch_size': args.test_batch_size}
164 if use_cuda:
165     cuda_kwargs = {'num_workers': 1,
166                   'pin_memory': True,
167                   'shuffle': True}
168     train_kwargs.update(cuda_kwargs)
169     test_kwargs.update(cuda_kwargs)
170
171 transform=transforms.Compose([
172     transforms.ToTensor(),
173     transforms.Normalize((0.1307,), (0.3081,))
174 ])
175 dataset1 = datasets.MNIST('../data', train=True, download=True,
176                           transform=transform)
177 dataset2 = datasets.MNIST('../data', train=False,
178                           transform=transform)
179 train_loader = torch.utils.data.DataLoader(dataset1, **train_kwargs)
180 test_loader = torch.utils.data.DataLoader(dataset2, **test_kwargs)
181
182 model = Net().to(device)
183 #####
184 # Initialize the Adadelata optimizer
185 optimizer = optim.Adadelta(model.parameters(), lr=args.lr)
186 #####
187
188 scheduler = StepLR(optimizer, step_size=1, gamma=args.gamma)
189 for epoch in range(1, args.epochs + 1):
190     train(args, model, device, train_loader, optimizer, epoch)
191     test(model, device, test_loader)
192     scheduler.step()
193
194 if args.save_model:
195     torch.save(model.state_dict(), "mnist_cnn.pt")
196
197
198 if __name__ == '__main__':
199     main()

```

1.2 Please report the **average loss** and **accuracy** on the test set after the 0th, 5th, and 10th epochs using the given hyperparameters. **(12 points, Extra Credits)**

The average loss and accuracy on the test set after the 0th, 5th, and 10th epochs using the given hyperparameters are as follows:

- **Epoch 1:** Test set: Average loss: 0.0549, Accuracy: 9814/10000 (98%)
- **Epoch 6:** Test set: Average loss: 0.0289, Accuracy: 9901/10000 (99%)
- **Epoch 11:** Test set: Average loss: 0.0276, Accuracy: 9910/10000 (99%)

2 Clustering

2.1 K-means Clustering (14 points)

1. **(6 Points)** Given n observations $X_1^n = \{X_1, \dots, X_n\}$, $X_i \in \mathcal{X}$, the K-means objective is to find k ($< n$) centres $\mu_1^k = \{\mu_1, \dots, \mu_k\}$, and a rule $f : \mathcal{X} \rightarrow \{1, \dots, K\}$ so as to minimize the objective

$$J(\mu_1^K, f; X_1^n) = \sum_{i=1}^n \sum_{k=1}^K \mathbb{1}(f(X_i) = k) \|X_i - \mu_k\|^2 \quad (1)$$

Let $\mathcal{J}_K(X_1^n) = \min_{\mu_1^K, f} J(\mu_1^K, f; X_1^n)$. Prove that $\mathcal{J}_K(X_1^n)$ is a non-increasing function of K .

Proof: When $K = 1$, the objective function is:

$$J(\mu_1, f; X_1^n) = \sum_{i=1}^n \|X_i - \mu_1\|^2$$

where μ_1 is the centroid of all points. This is the optimal solution when only one center is available, and it minimizes the total squared distance from each point to μ_1 .

For $K > 1$, the goal is to find $\mu_1^K = \{\mu_1, \mu_2, \dots, \mu_K\}$ and the assignment rule f that minimize the objective function. The function can be decomposed as follows:

$$J(\mu_1^K, f; X_1^n) = \sum_{i=1}^n \|X_i - \mu_{f(X_i)}\|^2$$

where $\mu_{f(X_i)}$ denotes the center assigned to X_i , and $f(X_i) \in \{1, 2, \dots, K\}$.

When we increase K to $K + 1$, the new objective function becomes:

$$J(\mu_1^{K+1}, f; X_1^n) = \sum_{i=1}^n \|X_i - \mu_{f'(X_i)}\|^2$$

where $\mu_{f'(X_i)}$ is the new center assigned to each point. Here, $f'(X_i) \in \{1, 2, \dots, K + 1\}$ is the new assignment rule, and the centers μ_1^{K+1} are optimized over the new assignment.

When K increases, we are effectively creating more clusters. By doing so, each point X_i can be assigned to a closer centroid, reducing the squared distance $\|X_i - \mu_k\|^2$ for some i . The new objective function $J(\mu_1^{K+1}, f; X_1^n)$ cannot be larger than the previous one because the optimization over the centers and assignments can only allow points to be assigned to closer centroids or stay the same. Therefore:

$$J(\mu_1^{K+1}, f; X_1^n) \leq J(\mu_1^K, f; X_1^n)$$

Since increasing K results in a partitioning of the data into more clusters and allows a finer optimization, the objective function $J(\mu_1^K, f; X_1^n)$ cannot increase as K increases. Hence, $\mathcal{J}_K(X_1^n)$ is a non-increasing function of K .

□

2. **(8 Points)** Consider the K-means (Lloyd's) clustering algorithm we studied in class. We terminate the algorithm when there are no changes to the objective. Show that the algorithm terminates in a finite number of steps.

Hint: You could try to prove that the objective decreases strictly, and the number of possible partitions is finite.

Proof: We begin by considering the K-means objective function:

$$J(\mu_1^K, f; X_1^n) = \sum_{i=1}^n \sum_{k=1}^K \mathbb{1}(f(X_i) = k) \|X_i - \mu_k\|^2$$

where $f(X_i)$ is the cluster assignment for point X_i , and μ_k is the centroid of cluster k . The algorithm proceeds iteratively by alternating between two steps:

For each point X_i , the algorithm assigns it to the cluster whose centroid is closest (minimizing $\|X_i - \mu_k\|^2$).

For each cluster k , the centroid μ_k is updated to the mean of the points assigned to that cluster.

We now show that the algorithm terminates in a finite number of steps by proving two key facts:

- The objective function decreases strictly with each iteration.
- The number of possible partitions of the data is finite.

At each iteration of the algorithm, the objective function $J(\mu_1^K, f; X_1^n)$ either stays the same or decreases. This can be seen from the following two facts:

- In the Cluster Assignment Step, each point X_i is assigned to the closest centroid μ_k . This means the new assignment minimizes the squared distance between X_i and its assigned centroid, and thus the objective function can either stay the same or decrease.
- In the Centroid Update Step, the new centroid μ_k is the mean of the points assigned to cluster k . The mean minimizes the sum of squared distances between the points and the centroid. Therefore, the centroid update step also results in a decrease or no change in the objective function.

Thus, the objective function $J(\mu_1^K, f; X_1^n)$ decreases strictly in each iteration, unless the current assignment and centroid values already minimize the objective. Therefore, the algorithm is guaranteed to make progress towards minimizing the objective until it converges.

Finiteness of the number of possible partitions:

- The K-means algorithm proceeds by assigning each point to one of the K clusters. There are n data points, and each point can belong to one of the K clusters. Therefore, the total number of possible assignments of the points to the clusters is finite and can be at most K^n (since each point has K choices for its assignment).

Since each iteration of the algorithm either keeps the same cluster assignments or results in a new set of assignments, and there are only a finite number of possible assignments, the algorithm can only cycle through a finite number of distinct configurations of cluster assignments.

Because the objective function decreases strictly in each iteration, and the number of possible distinct cluster assignments is finite, the algorithm must eventually reach a point where the cluster assignments no longer change. At this point, the algorithm terminates. Thus, the K-means algorithm terminates in a finite number of steps.

□

2.2 Experiment (20 Points)

In this question, we will evaluate K-means clustering and GMM on a simple 2 dimensional problem. First, create a two-dimensional synthetic dataset of 300 points by sampling 100 points each from the three Gaussian distributions shown below:

$$P_a = \mathcal{N}\left(\begin{bmatrix} -1 \\ -1 \end{bmatrix}, \sigma \begin{bmatrix} 2 & 0.5 \\ 0.5 & 1 \end{bmatrix}\right), \quad P_b = \mathcal{N}\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}, \sigma \begin{bmatrix} 1 & -0.5 \\ -0.5 & 2 \end{bmatrix}\right), \quad P_c = \mathcal{N}\left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}, \sigma \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}\right)$$

Here, σ is a parameter we will change to produce different datasets.

First implement K-means clustering and the expectation maximization algorithm for GMMs, with $K = 3$. Execute both methods on five synthetic datasets, generated as shown above with $\sigma \in \{0.5, 1, 2, 4, 8\}$. Finally, evaluate both methods on (i) the clustering objective (1) and (ii) the clustering accuracy. For each of the two criteria, plot the value achieved by each method against σ .

Guidelines:

- Both algorithms are only guaranteed to find only a local optimum so we recommend trying multiple restarts and picking the one with the lowest objective value (This is (1) for K-means and the negative log likelihood for GMMs). You may also experiment with a smart initialization strategy (such as kmeans++).
- To plot the clustering accuracy, you may treat the 'label' of points generated from distribution P_u as u , where $u \in \{a, b, c\}$. Assume that the cluster id i returned by a method is $i \in \{1, 2, 3\}$. Since clustering is an unsupervised learning problem, you should obtain the best possible mapping from $\{1, 2, 3\}$ to $\{a, b, c\}$ to compute the clustering objective. One way to do this is to compare the clustering centers returned by the method (centroids for K-means, means for GMMs) and map them to the distribution with the closest mean.

Points break down: 7 points each for implementation of each method, 6 points for reporting of evaluation metrics.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.stats import multivariate_normal
4 from sklearn.metrics import pairwise_distances_argmin
5 from itertools import permutations
6
7 def generate_data(sigma):
8     n_points = 100
9     cov_a = sigma * np.array([[2, 0.5], [0.5, 1]])
10    cov_b = sigma * np.array([[1, -0.5], [-0.5, 2]])
11    cov_c = sigma * np.array([[1, 0], [0, 2]])
12
13    points_a = np.random.multivariate_normal(mean=[-1, -1], cov=cov_a, size=n_points)
14    points_b = np.random.multivariate_normal(mean=[1, -1], cov=cov_b, size=n_points)
15    points_c = np.random.multivariate_normal(mean=[0, 1], cov=cov_c, size=n_points)
16
17    data = np.vstack([points_a, points_b, points_c])
18    labels = np.array([0] * n_points + [1] * n_points + [2] * n_points)
19    return data, labels
20
21 def kmeans(data, K, max_iter=100, n_restarts=10):
22     best_inertia = np.inf

```

```

23 best_centroids = None
24 best_labels = None
25
26 for _ in range(n_restarts):
27     centroids = data[np.random.choice(data.shape[0], K, replace=False)]
28     for _ in range(max_iter):
29         labels = pairwise_distances_argmin(data, centroids)
30
31         new_centroids = np.array([data[labels == k].mean(axis=0) for k in
32                                   ↪ range(K)])
33
34         if np.all(centroids == new_centroids):
35             break
36         centroids = new_centroids
37
38         inertia = sum(np.linalg.norm(data[labels == k] - centroids[k])**2 for k in
39                       ↪ range(K))
40         if inertia < best_inertia:
41             best_inertia = inertia
42             best_centroids = centroids
43             best_labels = labels
44
45     return best_centroids, best_labels
46
47 def gmm_em(data, K, max_iter=100, n_restarts=10):
48     n, d = data.shape
49     best_nll = np.inf
50     best_params = None
51
52     for _ in range(n_restarts):
53
54         weights = np.ones(K) / K
55         means = data[np.random.choice(n, K, replace=False)]
56         covariances = np.array([np.cov(data.T) for _ in range(K)])
57         responsibilities = np.zeros((n, K))
58
59         for _ in range(max_iter):
60
61             for k in range(K):
62                 responsibilities[:, k] = weights[k] * multivariate_normal.pdf(data,
63                                       ↪ mean=means[k], cov=covariances[k])
64             responsibilities /= responsibilities.sum(axis=1, keepdims=True)
65
66             N_k = responsibilities.sum(axis=0)
67             weights = N_k / n
68             means = np.array([np.sum(responsibilities[:, k][:, np.newaxis] * data,
69                                       ↪ axis=0) / N_k[k] for k in range(K)])
70             covariances = np.array([
71                 (responsibilities[:, k][:, np.newaxis] * (data - means[k])).T @ (data
72                                       ↪ - means[k]) / N_k[k]
73                 for k in range(K)
74             ])
75
76             nll = -np.sum(np.log(np.sum([
77                 weights[k] * multivariate_normal.pdf(data, mean=means[k],
78                                       ↪ cov=covariances[k])
79                 for k in range(K)
80             ], axis=0)))

```

```

75     if nll < best_nll:
76         best_nll = nll
77         best_params = (weights, means, covariances)
78
79     return best_params

```

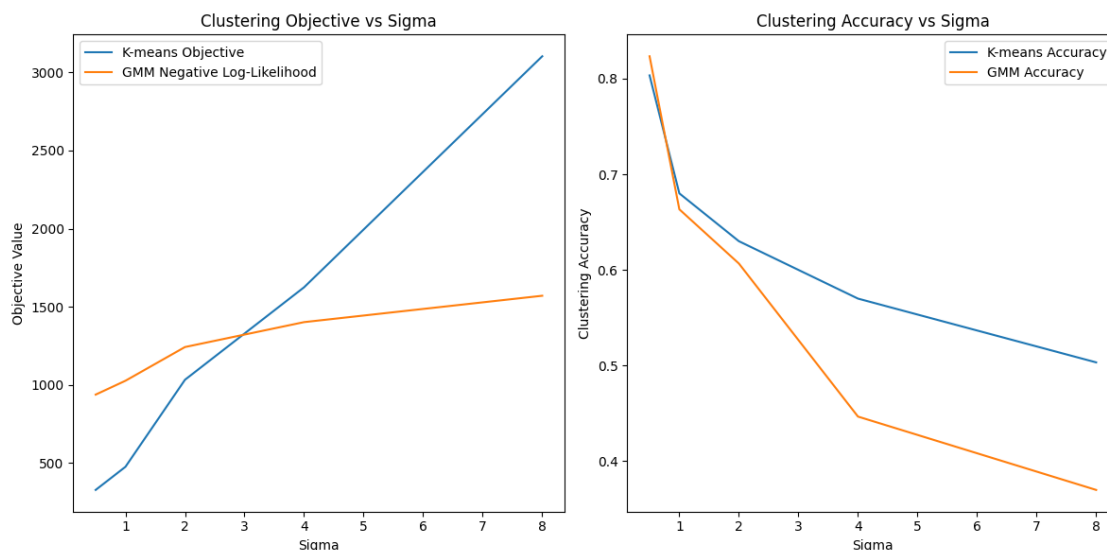


Figure 1: Clustering Objective and Accuracy v/s Sigma Plots

3 Linear Dimensionality Reduction

3.1 Principal Components Analysis (10 points)

Principal Components Analysis (PCA) is a popular method for linear dimensionality reduction. PCA attempts to find a lower dimensional subspace such that when you project the data onto the subspace as much of the information is preserved. Say we have data $X = [x_1^T; \dots; x_n^T] \in \mathbb{R}^{n \times D}$ where $x_i \in \mathbb{R}^D$. We wish to find a d ($< D$) dimensional subspace $A = [a_1, \dots, a_d] \in \mathbb{R}^{D \times d}$, such that $a_i \in \mathbb{R}^D$ and $A^T A = I_d$, so as to maximize $\frac{1}{n} \sum_{i=1}^n \|A^T x_i\|^2$.

1. **(4 Points)** Suppose we wish to find the first direction a_1 (such that $a_1^T a_1 = 1$) to maximize $\frac{1}{n} \sum_i (a_1^T x_i)^2$. Show that a_1 is the first right singular vector of X .

Hint: Write out the matrix form of the objective.

To show that a_1 is the first right singular vector of X , we start by considering the optimization problem:

$$\max_{a_1 \in \mathbb{R}^D, a_1^T a_1 = 1} \frac{1}{n} \sum_{i=1}^n (a_1^T x_i)^2.$$

This can be expressed in matrix form as:

$$\max_{a_1 \in \mathbb{R}^D, a_1^T a_1 = 1} \frac{1}{n} \|X a_1\|^2,$$

where $X \in \mathbb{R}^{n \times D}$ is the data matrix. Expanding $\|Xa_1\|^2$, we get:

$$\|Xa_1\|^2 = a_1^\top X^\top X a_1.$$

Thus, the optimization problem becomes:

$$\max_{a_1 \in \mathbb{R}^D, a_1^\top a_1 = 1} a_1^\top X^\top X a_1.$$

This is a Rayleigh quotient, and it is maximized when a_1 is the eigenvector of $X^\top X$ corresponding to its largest eigenvalue.

Now, recall that the singular value decomposition (SVD) of X is:

$$X = U\Sigma V^\top,$$

where:

- $U \in \mathbb{R}^{n \times n}$ is an orthogonal matrix containing the left singular vectors,
- $\Sigma \in \mathbb{R}^{n \times D}$ is a diagonal matrix with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq 0$ (where r is the rank of X),
- $V \in \mathbb{R}^{D \times D}$ is an orthogonal matrix containing the right singular vectors.

The matrix $X^\top X$ can be expressed in terms of the SVD as:

$$X^\top X = (U\Sigma V^\top)^\top (U\Sigma V^\top) = V\Sigma^\top U^\top U\Sigma V^\top = V\Sigma^\top \Sigma V^\top.$$

Since $\Sigma^\top \Sigma$ is a diagonal matrix with the squared singular values σ_i^2 on the diagonal, the eigenvectors of $X^\top X$ are the columns of V , and the eigenvalues are $\sigma_1^2, \sigma_2^2, \dots, \sigma_r^2$.

Therefore, the first principal direction a_1 that maximizes $a_1^\top X^\top X a_1$ is the eigenvector corresponding to the largest eigenvalue σ_1^2 . This eigenvector is the first right singular vector of X , denoted by v_1 , i.e.,

$$a_1 = v_1.$$

2. **(6 Points)** Given $a_1, \dots, a_k$, let $A_k = [a_1, \dots, a_k]$ and $\tilde{x}_i = x_i - A_k A_k^\top x_i$. We wish to find a_{k+1} , to maximize $\frac{1}{n} \sum_i (a_{k+1}^\top \tilde{x}_i)^2$. Show that a_{k+1} is the $(k+1)^{th}$ right singular vector of X .

Hint: You could use induction. If you use a reference, give a source. This will **not** cause a score deduction.

Given $a_1, \dots, a_k$, let $A_k = [a_1, \dots, a_k]$ and $\tilde{x}_i = x_i - A_k A_k^\top x_i$. We wish to find a_{k+1} such that it maximizes:

$$\frac{1}{n} \sum_{i=1}^n (a_{k+1}^\top \tilde{x}_i)^2.$$

- Base Case ($k = 1$): We have already shown that the first principal component a_1 is the first right singular vector of X . This means:

$$a_1 = v_1,$$

where v_1 is the first right singular vector of X .

- Inductive Step: Assume $a_1, a_2, \dots, a_k$ are the first k right singular vectors of X , i.e.,

$$A_k = [v_1, v_2, \dots, v_k],$$

where $v_1, v_2, \dots, v_k$ are the first k columns of V from the singular value decomposition $X = U\Sigma V^\top$.

Now, consider the projection $A_k A_k^\top x_i$ of x_i onto the subspace spanned by $\{a_1, \dots, a_k\}$. The residual vector $\tilde{x}_i$ is given by:

$$\tilde{x}_i = x_i - A_k A_k^\top x_i.$$

Our goal is to find a_{k+1} that maximizes:

$$\frac{1}{n} \sum_{i=1}^n (a_{k+1}^\top \tilde{x}_i)^2.$$

Expanding $\tilde{x}_i$, we have:

$$\tilde{x}_i = (I - A_k A_k^\top) x_i,$$

where I is the identity matrix. The optimization problem becomes:

$$\max_{a_{k+1} \in \mathbb{R}^D, a_{k+1}^\top a_{k+1} = 1} \frac{1}{n} \sum_{i=1}^n (a_{k+1}^\top (I - A_k A_k^\top) x_i)^2.$$

In matrix form, this can be expressed as:

$$\max_{a_{k+1} \in \mathbb{R}^D, a_{k+1}^\top a_{k+1} = 1} \|(I - A_k A_k^\top) X a_{k+1}\|^2.$$

Expanding $\|(I - A_k A_k^\top) X a_{k+1}\|^2$, we get:

$$\|(I - A_k A_k^\top) X a_{k+1}\|^2 = a_{k+1}^\top X^\top (I - A_k A_k^\top) X a_{k+1}.$$

Since $A_k^\top A_k = I_k$ (orthonormal columns), $(I - A_k A_k^\top)$ is a projection matrix onto the orthogonal complement of the subspace spanned by $\{a_1, \dots, a_k\}$.

The optimization problem:

$$\max_{a_{k+1} \in \mathbb{R}^D, a_{k+1}^\top a_{k+1} = 1} a_{k+1}^\top X^\top (I - A_k A_k^\top) X a_{k+1}$$

is solved when a_{k+1} is the eigenvector of $X^\top (I - A_k A_k^\top) X$ corresponding to the largest eigenvalue. This corresponds to the $(k+1)^{th}$ right singular vector v_{k+1} of X .

Therefore:

$$a_{k+1} = v_{k+1},$$

where v_{k+1} is the $(k+1)^{th}$ right singular vector of X , completing the induction.

3.2 Dimensionality reduction via optimization (22 points)

We will now motivate the dimensionality reduction problem from a slightly different perspective. The resulting algorithm has many similarities to PCA. We will refer to method as DRO.

As before, you are given data $\{x_i\}_{i=1}^n$, where $x_i \in \mathbb{R}^D$. Let $X = [x_1^\top; \dots; x_n^\top] \in \mathbb{R}^{n \times D}$. We suspect that the data actually lies approximately in a d dimensional affine subspace. Here $d < D$ and $d < n$. Our goal, as in PCA, is to use this dataset to find a d dimensional representation z for each $x \in \mathbb{R}^D$. (We will assume that the span of the data has dimension larger than d , but our method should work whether $n > D$ or $n < D$.)

Let $z_i \in \mathbb{R}^d$ be the lower dimensional representation for x_i and let $Z = [z_1^\top; \dots; z_n^\top] \in \mathbb{R}^{n \times d}$. We wish to find parameters $A \in \mathbb{R}^{D \times d}$, $b \in \mathbb{R}^D$ and the lower dimensional representation $Z \in \mathbb{R}^{n \times d}$ so as to minimize

$$J(A, b, Z) = \frac{1}{n} \sum_{i=1}^n \|x_i - Az_i - b\|^2 = \|X - ZA^\top - \mathbf{1}b^\top\|_F^2. \quad (2)$$

Here, $\|A\|_F^2 = \sum_{i,j} A_{ij}^2$ is the Frobenius norm of a matrix.

1. **(3 Points)** Let $M \in \mathbb{R}^{d \times d}$ be an arbitrary invertible matrix and $p \in \mathbb{R}^d$ be an arbitrary vector. Denote, $A_2 = A_1 M^{-1}$, $b_2 = b_1 - A_1 M^{-1}p$ and $Z_2 = Z_1 M^\top + \mathbf{1}p^\top$. Show that both (A_1, b_1, Z_1) and (A_2, b_2, Z_2) achieve the same objective value J (2).

Therefore, in order to make the problem determined, we need to impose some constraint on Z . We will assume that the z_i 's have zero mean and identity covariance. That is,

$$\bar{Z} = \frac{1}{n} \sum_{i=1}^n z_i = \frac{1}{n} Z^\top \mathbf{1}_n = 0, \quad S = \frac{1}{n} \sum_{i=1}^n z_i z_i^\top = \frac{1}{n} Z^\top Z = I_d$$

Here, $\mathbf{1}_d = [1, 1, \dots, 1]^\top \in \mathbb{R}^d$ and I_d is the $d \times d$ identity matrix.

We aim to show that the objective value J remains unchanged for the transformations:

$$A_2 = A_1 M^{-1}, \quad b_2 = b_1 - A_1 M^{-1}p, \quad Z_2 = Z_1 M^\top + \mathbf{1}p^\top,$$

where $M \in \mathbb{R}^{d \times d}$ is invertible, and $p \in \mathbb{R}^d$ is an arbitrary vector.

The objective function is given as:

$$J(A, b, Z) = \|X - ZA^\top - \mathbf{1}b^\top\|_F^2.$$

Using the transformed parameters (A_2, b_2, Z_2) , the term inside the Frobenius norm becomes:

$$X - Z_2 A_2^\top - \mathbf{1}b_2^\top.$$

Substitute $A_2 = A_1 M^{-1}$, $b_2 = b_1 - A_1 M^{-1}p$, and $Z_2 = Z_1 M^\top + \mathbf{1}p^\top$:

$$Z_2 A_2^\top = (Z_1 M^\top + \mathbf{1}p^\top)(A_1 M^{-1})^\top = Z_1 M^\top (M^{-1})^\top A_1^\top + \mathbf{1}p^\top (M^{-1})^\top A_1^\top,$$

and

$$\mathbf{1}b_2^\top = \mathbf{1}(b_1 - A_1 M^{-1}p)^\top = \mathbf{1}b_1^\top - \mathbf{1}p^\top (M^{-1})^\top A_1^\top.$$

Thus, the expression becomes:

$$X - Z_2 A_2^\top - \mathbf{1}b_2^\top = X - Z_1 A_1^\top - \mathbf{1}b_1^\top.$$

The above shows that the transformed parameters (A_2, b_2, Z_2) result in the same residual as the original parameters (A_1, b_1, Z_1) . Therefore, the objective value remains unchanged:

$$J(A_2, b_2, Z_2) = \|X - Z_2 A_2^\top - \mathbf{1}b_2^\top\|_F^2 = \|X - Z_1 A_1^\top - \mathbf{1}b_1^\top\|_F^2 = J(A_1, b_1, Z_1).$$

2. **(16 Points)** Outline a procedure to solve the above problem. You only need to specify how you would obtain A, Z, b which minimize the objective and satisfy the constraints.

Hint: You can consider b first. The rank k approximation of a matrix in Frobenius norm is obtained by taking its SVD and then zeroing out all but the first k singular values. You could search for Eckart–Young–Mirsky theorem for reference.

We aim to minimize the objective function

$$J(A, b, Z) = \|X - ZA^\top - \mathbf{1}b^\top\|_F^2,$$

subject to the constraints:

$$\frac{1}{n}Z^\top \mathbf{1} = 0 \quad (\text{zero mean}), \quad \frac{1}{n}Z^\top Z = I_d \quad (\text{identity covariance}).$$

Minimize J with respect to b while keeping A and Z fixed. The problem becomes:

$$\min_b \|X - ZA^\top - \mathbf{1}b^\top\|_F^2.$$

For a fixed A and Z , the optimal b is obtained by solving:

$$\frac{\partial J}{\partial b} = 0 \implies b = \frac{1}{n}X^\top \mathbf{1} - \frac{1}{n}A^\top Z^\top \mathbf{1}.$$

Since $\frac{1}{n}Z^\top \mathbf{1} = 0$ (from the zero-mean constraint), this simplifies to:

$$b = \frac{1}{n}X^\top \mathbf{1}.$$

Using the optimal b , center the data matrix:

$$\tilde{X} = X - \mathbf{1}b^\top = X - \frac{1}{n}\mathbf{1}\mathbf{1}^\top X.$$

Here, $\tilde{X}$ is the mean-centered version of X .

The Eckart-Young-Mirsky theorem tells us that the rank- d approximation of a matrix in Frobenius norm is obtained by its truncated SVD. Compute the SVD of the centered data:

$$\tilde{X} = U\Sigma V^\top,$$

where $U \in \mathbb{R}^{n \times n}$, $\Sigma \in \mathbb{R}^{n \times D}$, and $V \in \mathbb{R}^{D \times D}$ are the singular value decomposition components.

Take the first d singular values and corresponding singular vectors:

$$\tilde{U}_d \in \mathbb{R}^{n \times d}, \quad \tilde{\Sigma}_d \in \mathbb{R}^{d \times d}, \quad \tilde{V}_d \in \mathbb{R}^{D \times d}.$$

Using the truncated SVD, define:

$$Z = \sqrt{n}\tilde{U}_d, \quad A = \tilde{V}_d\tilde{\Sigma}_d/\sqrt{n}.$$

This choice ensures that Z satisfies the constraints:

$$\frac{1}{n}Z^\top \mathbf{1} = 0, \quad \frac{1}{n}Z^\top Z = I_d.$$

The parameters minimizing $J(A, b, Z)$ are:

$$b = \frac{1}{n}X^\top \mathbf{1}, \quad Z = \sqrt{n}\tilde{U}_d, \quad A = \tilde{V}_d\tilde{\Sigma}_d/\sqrt{n}.$$

3. **(3 Points)** You are given a point x_* in the original D dimensional space. State the rule to obtain the d dimensional representation z_* for this new point. (If x_* is some original point x_i from the D -dimensional space, it should be the d -dimensional representation z_i .)

Given a point $x_* \in \mathbb{R}^D$, the goal is to find its d -dimensional representation $z_* \in \mathbb{R}^d$ in the reduced space.

The relationship between the high-dimensional point x_* and its low-dimensional representation z_* is:

$$x_* = Az_* + b,$$

where $A \in \mathbb{R}^{D \times d}$ and $b \in \mathbb{R}^D$ are the parameters learned during the dimensionality reduction process.

Rearranging for z_* , we have:

$$z_* = A^\top(x_* - b).$$

If $x_* = x_i$, one of the original data points, then the computed z_* will correspond to the d -dimensional representation z_i of that point:

$$z_* = z_i.$$

For any new point $x_* \in \mathbb{R}^D$, its d -dimensional representation z_* is computed as:

$$z_* = A^\top(x_* - b).$$

3.3 Experiment (34 points)

Here we will compare the above three methods on two data sets.

- We will implement three variants of PCA:
 1. "buggy PCA": PCA applied directly on the matrix X .
 2. "demeaned PCA": We subtract the mean along each dimension before applying PCA.
 3. "normalized PCA": Before applying PCA, we subtract the mean and scale each dimension so that the sample mean and standard deviation along each dimension is 0 and 1 respectively.
- One way to study how well the low dimensional representation Z captures the linear structure in our data is to project Z back to D dimensions and look at the reconstruction error. For PCA, if we mapped it to d dimensions via $z = Vx$ then the reconstruction is $V^\top z$. For the preprocessed versions, we first do this and then reverse the preprocessing steps as well. For DRO we just compute $Az + b$. We will compare all methods by the reconstruction error on the datasets.
- Please implement code for the methods: Buggy PCA (just take the SVD of X) , Demeaned PCA, Normalized PCA, DRO. In all cases your function should take in an $n \times d$ data matrix and d as an argument. It should return the the d dimensional representations, the estimated parameters, and the reconstructions of these representations in D dimensions.

Hint: You can use `numpy.linalg.svd` for SVD.

- You are given two datasets: A two Dimensional dataset with 50 points `data2D.csv` and a thousand dimensional dataset with 500 points `data1000D.csv`.
- For the 2D dataset use $d = 1$. For the 1000D dataset, you need to choose d . For this, observe the singular values in DRO and see if there is a clear "knee point" in the spectrum. Attach any figures/Statistics you computed to justify your choice.
- For the 2D dataset you need to attach the a plot comparing the original points with the reconstructed points for all 4 methods. For both datasets you should also report the reconstruction errors, that is the squared sum of differences $\sum_{i=1}^n \|x_i - r(z_i)\|^2$, where x_i 's are the original points and $r(z_i)$ are the D dimensional points reconstructed from the d dimensional representation z_i .

- **Questions:** After you have completed the experiments, please answer the following questions.

1. Look at the results for Buggy PCA. The reconstruction error is bad and the reconstructed points don't seem to well represent the original points. Why is this?

Hint: Which subspace is Buggy PCA trying to project the points onto?

Answer: The Buggy PCA method does not perform any preprocessing steps (such as centering the data by subtracting the mean), and directly applies PCA on the raw data. This means that Buggy PCA projects the data onto the subspace defined by the top singular vectors of the raw data matrix. However, this is problematic because the raw data may not be centered around the origin, and the projection may not properly align with the intrinsic structure of the data. As a result, the reconstructed points may not represent the original data well, leading to high reconstruction errors.

2. The error criterion we are using is the average squared error between the original points and the reconstructed points. In both examples DRO and demeaned PCA achieves the lowest error among all methods. Is this surprising? Why?

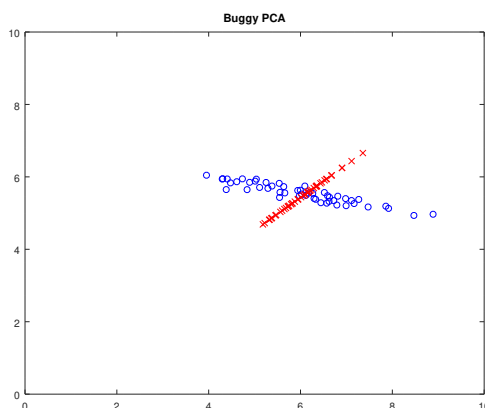
Answer: This result is not surprising. Both DRO (Dimensionality Reduction via Optimization) and demeaned PCA (which subtracts the mean of the data before applying PCA) are designed to capture the intrinsic structure of the data more effectively by centering the data before dimensionality reduction. By removing the mean, these methods focus on the variation of the data, which leads to more accurate representations in the lower-dimensional space. In contrast, Buggy PCA and normalized PCA may be less effective due to issues like lack of proper centering or over-scaling, resulting in higher reconstruction errors.

- Point allocation:

- Implementation of the three PCA methods: **(6 Points)**
- Implementation of DRO: **(6 points)**
- Plots showing original points and reconstructed points for 2D dataset for each one of the 4 methods: **(10 points)**
- Implementing reconstructions and reporting results for each one of the 4 methods for the 2 datasets: **(5 points)**
- Choice of d for 1000D dataset and appropriate justification: **(3 Points)**
- Questions **(4 Points)**

Answer format:

The graph bellow is in example of how a plot of one of the algorithms for the 2D dataset may look like:



The blue circles are from the original dataset and the red crosses are the reconstructed points.

And this is how the reconstruction error may look like for Buggy PCA for the 2D dataset: 0.886903

Implementations for Buggy PCA, Demeaned PCA, Normalized PCA and DRO:

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def pca(X, d, preprocess=None):
5      if preprocess == "demean":
6          mean = X.mean(axis=0)
7          X_prep = X - mean
8      elif preprocess == "normalize":
9          mean = X.mean(axis=0)
10         std = X.std(axis=0, ddof=1)
11         X_prep = (X - mean) / std
12     else:
13         mean, std = None, None
14         X_prep = X.copy()
15
16     U, S, VT = np.linalg.svd(X_prep, full_matrices=False)
17     V = VT.T[:, :d]
18     Z = X_prep @ V
19
20
21     X_reconstructed = Z @ V.T
22     if preprocess == "demean":
23         X_reconstructed += mean
24     elif preprocess == "normalize":
25         X_reconstructed = X_reconstructed * std + mean
26
27     return Z, V, X_reconstructed
28
29
30 def buggy_pca(X, d):
31     return pca(X, d, preprocess=None)
32
33 def demeaned_pca(X, d):
34     return pca(X, d, preprocess="demean")
35
36 def normalized_pca(X, d):
37     return pca(X, d, preprocess="normalize")
38
39 def dro(X, d):
40     n, D = X.shape
41     b = X.mean(axis=0)
42
43     X_centered = X - b
44
45     U, S, VT = np.linalg.svd(X_centered, full_matrices=False)
46
47     A = VT[:, :d].T
48     Z = X_centered @ A
49
50     X_reconstructed = Z @ A.T + b
51
52     return Z, A, b, X_reconstructed, S

```

Plots showing original points and reconstructed points for 2D dataset for each one of the 4 methods:

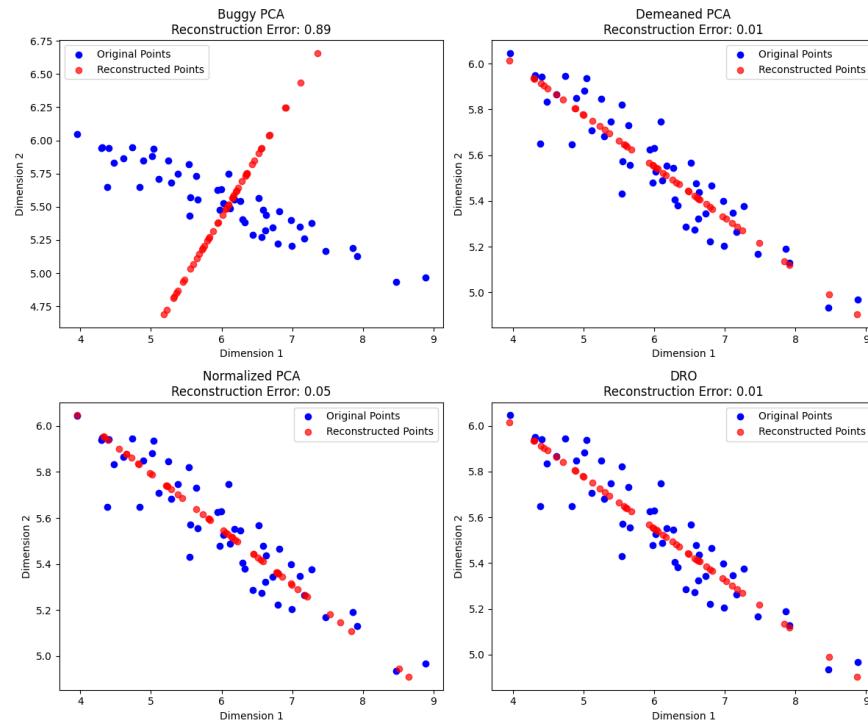


Figure 2: Plots showing original points and reconstructed points for 2D dataset for each one of the 4 methods

Implementing reconstructions and reporting results for each one of the 4 methods for the 2 datasets:

2D Dataset:

Buggy PCA Reconstruction Error: 0.8869030837347942

Demeaned PCA Reconstruction Error: 0.01000608562851292

Normalized PCA Reconstruction Error: 0.04947208345477067

DRO Reconstruction Error: 0.01000608562851292

1000D Dataset:

Buggy PCA Reconstruction Error (1000D): 802.7313986203584

Demeaned PCA Reconstruction Error (1000D): 273.0459589786028

Normalized PCA Reconstruction Error (1000D): 273.6285099762326

DRO Reconstruction Error (1000D): 273.0459589786028

Choice of d for 1000D dataset and appropriate justification:

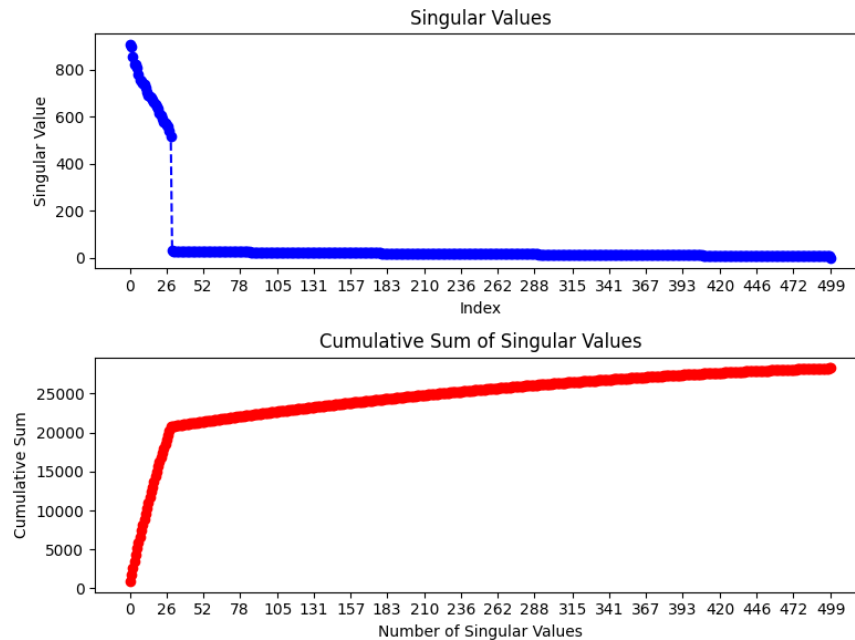


Figure 3: Plots of singular values and cumulative sum of singular values to find the knee point

Looking at the plot the optimal value of d should be 30. The knee point at 30 is justified by the behavior of the singular values, where the sharp drop transitions into a flat region, indicating that further dimensions don't significantly add more information.

Questions: After you have completed the experiments, please answer the following questions.

1. Look at the results for Buggy PCA. The reconstruction error is bad and the reconstructed points don't seem to well represent the original points. Why is this?

Hint: Which subspace is Buggy PCA trying to project the points onto?

Answer: The Buggy PCA method does not perform any preprocessing steps (such as centering the data by subtracting the mean), and directly applies PCA on the raw data. This means that Buggy PCA projects the data onto the subspace defined by the top singular vectors of the raw data matrix. However, this is problematic because the raw data may not be centered around the origin, and the projection may not properly align with the intrinsic structure of the data. As a result, the reconstructed points may not represent the original data well, leading to high reconstruction errors.

2. The error criterion we are using is the average squared error between the original points and the reconstructed points. In both examples DRO and demeaned PCA achieves the lowest error among all methods. Is this surprising? Why?

Answer: This result is not surprising. Both DRO (Dimensionality Reduction via Optimization) and demeaned PCA (which subtracts the mean of the data before applying PCA) are designed to capture the intrinsic structure of the data more effectively by centering the data before dimensionality reduction. By removing the mean, these methods focus on the variation of the data, which leads to more accurate representations in the lower-dimensional space. In contrast, Buggy PCA and normalized PCA may be less effective due to issues like lack of proper centering or over-scaling, resulting in higher reconstruction errors.